

Índice

Informe Técnico: Juego de Carreras 3D en OpenGL	1
1. Visión general del proyecto y librerías utilizadas	1
2. Organización del código y archivos principales	2
2.1 main.cpp	2
2.2 Geometría y geometria_coche.h	2
2.3 Shaders	2
3. Físicas del coche y controles	3
Dirección	3
Marchas	3
Asfalto y hierba	3
4. Sistema de nitro	4
5. Diseño del escenario	4
6. Interfaz del juego	4
7. Iluminación, texturas y efectos visuales	5
Cielo dinámico	5
Modo noche	5
Faros del coche	5
Fuego del nitro	5
8. Cámaras del juego	6
9. Sistema de vueltas y final de carrera	6
Conclusión	6

Informe Técnico: Juego de Carreras 3D en OpenGL

Este informe explica el funcionamiento del proyecto **Juego de Carreras**. El objetivo principal del proyecto es crear una experiencia de conducción tipo arcade, con una pista cerrada, físicas básicas de movimiento, sistema de vueltas, iluminación dinámica, cámaras y una interfaz en pantalla para mostrar información al jugador.

1. Visión general del proyecto y librerías utilizadas

El juego consiste en completar **dos vueltas** a un circuito cerrado en el menor tiempo posible. El jugador controla un coche con comportamiento arcade, es decir, no se busca una simulación totalmente realista, sino una conducción sencilla, rápida y visualmente clara.

El coche puede acelerar (W), frenar (S), girar(A,D), cambiar de marcha (UP, DOWN), usar nitro (N), salirse de la pista y volver a ella. Además, el escenario incluye cambios entre día y noche (T), faros (F), una cámara en distintas posiciones y un HUD que muestra datos importantes durante la carrera.

Para desarrollar el proyecto se han utilizado varias librerías:

- **OpenGL y GLAD:** comunicarse con la tarjeta gráfica y dibujar los objetos 3D en pantalla.
- **GLFW:** crear la ventana del juego y detectar la entrada del teclado.
- **GLM:** cálculos matemáticos necesarios para mover, rotar y escalar objetos en 3D.
- **stb_image:** cargar imágenes para usarlas como texturas.

- **Dear ImGui:** crear la interfaz del juego, como paneles de información, barras de progreso y datos del coche.
-

2. Organización del código y archivos principales

El proyecto está organizado de forma modular para separar las distintas partes del programa. Por un lado está la lógica principal del juego, por otro la geometría de los objetos, y por otro los shaders, que son los programas encargados de calcular cómo se ven los objetos en pantalla.

2.1 `main.cpp`

El archivo `main.cpp` es la parte central del proyecto. Se inicializa la ventana, se configuran las librerías, se cargan los recursos y se ejecuta el bucle principal del juego.

Una de sus funciones más importantes es el **bucle principal**, que se repite continuamente mientras el juego está abierto. En cada vuelta del bucle se actualizan las físicas del coche, se procesan las teclas pulsadas por el jugador, se dibuja el escenario y se actualiza la interfaz.

También se usa un sistema basado en `DeltaTime`, que mide el tiempo transcurrido entre fotogramas. Esto es importante porque evita que el coche se mueva más rápido o más lento dependiendo del rendimiento del ordenador. De esta forma, la velocidad y los giros se mantienen estables aunque cambien los FPS.

Además, en este archivo se controlan variables generales del juego, como el tiempo de carrera, el estado de noche o día, el uso del nitro, la posición del coche y el número de vueltas completadas.

2.2 Geometría y `geometria_coche.h`

El proyecto no carga modelos 3D externos complejos. En lugar de eso, los objetos se construyen usando figuras básicas como cubos, cilindros y esferas. Esto hace que el proyecto sea más ligero y permite entender mejor cómo se genera cada parte del escenario.

En el archivo `geometria_coche.h` se define la geometría base del coche y de algunos objetos. Por ejemplo, se utiliza un array de vértices para representar cubos. Cada vértice contiene información sobre su posición, su normal y sus coordenadas de textura.

Las normales son necesarias para que la iluminación funcione correctamente, ya que indican hacia dónde mira cada cara del objeto. Las coordenadas de textura sirven para colocar imágenes sobre las superficies.

También se genera la geometría de las ruedas mediante una función que crea cilindros. En vez de escribir todos los puntos manualmente, el código calcula la forma circular usando funciones trigonométricas como `sin` y `cos`. Así se consigue una rueda redonda formada por varios segmentos.

2.3 Shaders

Los shaders son una parte esencial del proyecto, controlan cómo se transforma y cómo se ilumina cada objeto en la GPU.

El **vertex shader** transforma los vértices de cada objeto. Los coloca en su posición local, después en el mundo 3D, luego en relación con la cámara y finalmente en la pantalla. Usa matrices de modelo, vista y proyección.

También calcula correctamente las normales de los objetos para que la luz no se deforme cuando una figura se escala o se rota.

El **fragment shader** trabaja a nivel de píxel. Decide el color final de cada punto visible en pantalla. Combina la textura del objeto, la iluminación, la transparencia y los efectos especiales.

En este shader se aplica un modelo de iluminación con luz ambiental, difusa y especular. Esto permite que los objetos no se vean planos, sino que reaccionen a la luz de forma más realista. También se incluyen focos tipo faro, con una dirección y un ángulo de apertura, para simular la luz que sale desde la parte delantera del coche.

Además, algunos objetos pueden ser emisivos. Esto significa que no dependen de la luz externa, sino que parecen emitir luz propia, como ocurre con el fuego del nitro.

3. Físicas del coche y controles

El movimiento del coche se gestiona principalmente en la función `actualizarFisicas()`. Esta parte del código calcula cómo cambia la posición, la velocidad y la orientación del coche según las teclas pulsadas y el estado actual del vehículo.

El sistema de conducción está pensado para ser arcade. El coche tiene inercia, puede derrapar ligeramente y pierde velocidad cuando sale del asfalto, pero no intenta ser una simulación física totalmente realista.

Dirección

El jugador puede girar usando las teclas **A** y **D**. El giro del volante tiene un límite, de manera que no se puede girar infinitamente. Cuando el jugador deja de pulsar la tecla, el volante vuelve automáticamente al centro.

Esto hace que el control sea más cómodo y evita que el coche se quede girando demasiado tiempo después de soltar la tecla.

Marchas

El coche tiene **seis marchas**, cada una con una velocidad máxima distinta. La primera marcha permite poca velocidad, mientras que la sexta permite alcanzar la velocidad máxima normal.

El jugador puede cambiar de marcha manualmente usando las flechas **UP** y **DOWN**. Además, si el coche pierde mucha velocidad, el sistema reduce automáticamente a una marcha inferior para adaptarse mejor a la velocidad actual.

Esto hace que el coche tenga una sensación más dinámica, aunque siga siendo fácil de manejar.

Asfalto y hierba

El circuito diferencia entre zonas de asfalto y zonas de hierba. Mientras el coche está dentro de la pista, puede alcanzar su velocidad normal. Sin embargo, si se sale del asfalto, la velocidad máxima

se reduce mucho.

Esto funciona como una penalización para el jugador. Salirse de la pista no detiene completamente el coche, pero sí hace que pierda mucho tiempo y sea más difícil mantener una buena vuelta.

4. Sistema de nitro

El juego incluye un sistema de **nitro**, activado con la tecla **N**. Esta función permite al coche ganar velocidad durante un corto periodo de tiempo.

Al usar el nitro, la velocidad del coche aumenta de forma inmediata y durante unos segundos se permite superar el límite normal de velocidad. También se reduce la penalización de la hierba, de modo que el jugador puede cruzar zonas fuera de pista con algo más de libertad.

Para evitar que el nitro se use constantemente, se ha añadido un tiempo de recarga. Después de activarlo, el jugador debe esperar varios segundos antes de poder volver a usarlo.

Este sistema añade una parte estratégica al juego, ya que el jugador debe decidir cuándo conviene usar el nitro: en una recta, para recuperar velocidad tras un error o para intentar mejorar el tiempo de vuelta.

5. Diseño del escenario

El escenario no está formado por un modelo 3D fijo cargado desde un archivo. En su lugar, se genera mediante código dentro de la función `dibujarEntorno()`.

El terreno se dibuja por secciones, colocando muchas piezas sobre el plano X/Z. Para cada sección, el programa calcula si esa zona pertenece al asfalto, a la hierba, a los muros o a la línea de meta.

La forma de la pista se calcula usando una función matemática que crea un recorrido ovalado con variaciones. Gracias a esto, el circuito no es simplemente un círculo perfecto, sino que tiene curvas más interesantes y zonas con cambios suaves.

El código también añade muros alrededor de la pista. Estos muros sirven para marcar visualmente los límites del circuito y ayudan a reforzar la sensación de estar dentro de un trazado cerrado.

La línea de meta se representa con una textura de cuadros, de forma que el jugador puede identificar fácilmente dónde empieza y termina cada vuelta.

6. Interfaz del juego

La interfaz se ha creado usando **Dear ImGui**. Esta librería permite dibujar paneles encima de la escena 3D sin tener que crear un sistema de menús desde cero.

En pantalla se muestran datos útiles para el jugador, como la velocidad, la marcha actual, el tiempo de carrera, el estado del nitro y otra información relacionada con la conducción.

Uno de los elementos más importantes es la barra de recarga del nitro. Esta barra muestra visualmente cuánto falta para poder volver a usarlo. Su valor se actualiza en tiempo real leyendo la variable interna del temporizador del nitro.

También se ha añadido información por consola para depuración. El programa imprime datos importantes del estado del coche y del juego sin llenar la terminal con demasiadas líneas. Para ello se actualiza la misma línea varias veces por segundo, lo que permite revisar valores durante la ejecución sin saturar la salida.

7. Iluminación, texturas y efectos visuales

El proyecto incluye varios efectos visuales para mejorar la apariencia del juego.

Cielo dinámico

El cielo se representa mediante una esfera grande que rodea la escena. Esta esfera se mueve junto con la cámara, por lo que da la sensación de estar siempre a mucha distancia.

Para evitar problemas visuales, se desactiva temporalmente la escritura en el buffer de profundidad mientras se dibuja el cielo. Así se consigue que actúe como fondo sin interferir con el resto de objetos de la escena.

Modo noche

El juego permite cambiar entre día y noche usando la tecla **T**. Cuando se activa el modo noche, la luz ambiental se reduce y el escenario se ve más oscuro.

Este cambio afecta al shader, que modifica la iluminación general de la escena. Así se consigue que el entorno tenga una apariencia diferente sin necesidad de cargar otro mapa o escenario.

Faros del coche

Los faros se activan con la tecla **F**. Funcionan como dos focos colocados en la parte delantera del coche. Su posición y dirección se actualizan según la posición y la rotación del vehículo.

Esto permite que la luz siga al coche correctamente y que ilumine la pista en la dirección hacia la que está mirando.

Fuego del nitro

Cuando se usa el nitro, aparece un efecto visual de fuego en la parte trasera del coche. Este fuego se marca como objeto emisor, por lo que se ve brillante aunque la escena esté oscura.

Además, su tamaño varía rápidamente usando una función basada en el tiempo. Esto crea un efecto de parpadeo que simula mejor una llama en movimiento.

8. Cámaras del juego

El simulador incluye varias cámaras que el jugador puede cambiar durante la partida.

La cámara de persecución sigue al coche desde atrás y desde cierta altura. Es la vista principal, ya que permite ver bien el vehículo y la pista.

También hay una cámara en primera persona, colocada cerca del coche para dar una sensación más cercana a la conducción desde dentro del vehículo.

Por último, existe una cámara cenital, situada muy por encima del escenario. Esta vista permite observar el circuito desde arriba. Además, puede activarse al terminar la carrera para mostrar mejor el resultado final.

Todas las cámaras se calculan usando `glm::lookAt`, indicando la posición de la cámara, el punto al que mira y la orientación vertical. La posición de cada cámara se actualiza en función de la posición y la rotación del coche.

9. Sistema de vueltas y final de carrera

El juego controla cuándo el coche cruza la línea de meta. Para evitar contar vueltas falsas, no basta con pasar cerca de la línea, sino que se comprueba también la dirección del coche y el momento exacto del cruce.

Cuando el coche cruza correctamente la meta, se actualiza el número de vueltas completadas. Al llegar a la segunda vuelta, la carrera termina y se muestra el resultado.

Este sistema permite que el circuito funcione como una carrera real de contrarreloj, donde el jugador debe completar el recorrido correctamente para registrar su tiempo final.

Conclusión

El proyecto **Simulador de Carreras 3D** combina varias partes importantes de la programación gráfica y de videojuegos. Incluye renderizado 3D con OpenGL, generación de geometría mediante código, físicas básicas de conducción, iluminación dinámica, texturas, interfaz en pantalla, cámaras múltiples y sistema de vueltas.

Aunque el estilo de conducción es arcade, el código incorpora bastantes elementos técnicos: uso de shaders, matrices de transformación, control de tiempo con `DeltaTime`, generación procedural del escenario, faros dinámicos, objetos emisivos y gestión de estados del juego.

En conjunto, el simulador demuestra cómo se puede construir un videojuego 3D funcional sin depender de un motor gráfico externo, controlando directamente tanto la lógica del juego como la parte visual. ““